Comprehensive Overview: SQL-Based Data Preparation Pipeline (502)

Executive Summary

The `502_working.ipynb` notebook provides an efficient, SQL-based data preparation pipeline that filters and prepares military officer snapshot data for Cox regression analysis. The pipeline leverages PostgreSQL for fast initial filtering before loading data into pandas, then performs data quality checks and calculates key demographic variables (year groups, dates of rank, prestige unit assignments) while preserving all officer snapshots throughout their entire career.

Core Concept: Use SQL for efficient early filtering and PID culling, then perform data quality checks and variable calculations in pandas, preserving all snapshots (all ranks) for complete career history analysis.

Pipeline Architecture

High-Level Flow

```
Raw Master Table (mv_master_ad_army_qtr_v3a)
    ↓
[CELL 0.5] Bad MOS Filtering (SQL-based PID culling)
    ├─ Load bad MOS codes (CSV or hardcoded)
    ├─ Create ref_cull_mos table
    ├─ Create pids_o table (all officer PIDs)
    ├─ Create ref_cull_pids table (PIDs with bad MOS)
    └─ Create pids_o_culld table (filtered PIDs) → SQL table
    ↓
[CELL 1] SQL Table Creation
    ├─ Create snapshots table (distinct snapshot dates)
    ├─ Create snap_window table (snapshots within date window)
    └─ Create b_502_base table (filtered base data) → SQL table
    ↓
[CELL 2] Load Base DataFrame
    └─ Load b_502_base from SQL → df_502_00_base.feather
    ↓
[CELL 3] Data Quality Checks
    ├─ Identify breaks in service
    ├─ Identify duplicate appointment dates
    ├─ Identify duplicate DOR dates
    └─ Exclude problematic officers → df_502_01_clean.feather
    ↓
[CELL 4] Calculate Year Groups (yg)
    └─ Calculate yg from appointment date → df_502_02_with_yg.feather
    ↓
[CELL 5] Calculate Date of Rank (dor_cpt and dor_maj)
    └─ Extract DOR from CPT/MAJ snapshots, map to ALL snapshots → df_502_03_with_dor.feather
    ↓
[CELL 6] Create Prestige Unit Variables
    ├─ Load UIC hierarchy data
    ├─ Create prestige_unit (time-varying binary)
    └─ Create prestige_sum (time-varying cumulative) → df_502_04_with_prestige.feather
    ↓
[CELL 7] Final Output
    └─ Verify columns and save → df_502_base.feather (for use in 520 pipeline)
```

Key Design Principles

1. **SQL-First Filtering**: Leverage PostgreSQL for efficient early filtering before loading into pandas
2. **Progressive Saves**: Each stage saves intermediate results using standard naming pattern: `df_502_XX_stage.feather`
3. **Selective Execution**: Boolean flags allow running only specific cells (e.g., skip early stages if intermediate files exist)
4. **Preserve All Snapshots**: Keeps all officer snapshots (all ranks) throughout the pipeline for complete career history
5. **Time-Varying Variables**: Prestige variables are mapped to all snapshots, allowing analysis of career progression
6. **Data Quality Focus**: Comprehensive checks for breaks in service, duplicate dates, and data inconsistencies

---

Pipeline Structure

Cell Execution Flags

Located in **CELL 0**, these boolean flags control which cells execute:

```
CELL0_5 = True  # Bad MOS Filtering (create pids_o_culld table)
CELL1 = True   # SQL Table Creation (snapshots, snap_window, base table)
CELL2 = True   # Load Base DataFrame from SQL
CELL3 = True   # Data Quality Checks (breaks in service, duplicate dates)
CELL4 = True   # Calculate Year Groups (yg)
CELL5 = True   # Calculate Date of Rank (dor_cpt and dor_maj)
CELL6 = True   # Create Prestige Unit Variables (prestige_unit and prestige_sum)
CELL7 = True   # Final Output (add columns and save df_502_base.feather)
```

Usage: Set flags to `False` to skip cells (e.g., if intermediate files already exist).

CELL 0: Setup & Imports

Purpose: Initialize the analysis environment with library imports, database connections, and global variables.

Key Components: - **Library Imports**: pandas, numpy, sqlalchemy, datetime, etc. - **Database Connection**: PostgreSQL connection string and SQLAlchemy engine - **Global Variables**: - `data_table_pde`: Master data table (`study_talent_net.mv_master_ad_army_qtr_v3a`) - `study_schema`: Schema for study tables (`study_talent_net.`) - `default_schema`: Schema for shared tables (`study_talent_net_shared.`) - `pids_table`: Reference to `pids_o_culld` table (created in CELL 0.5) - `window_tup`: Date window for snapshots `(2000, 2022)` - `col_list_base`: List of columns to include in base table - **Utility Functions**: `print_v()` for verbose output

Output: Environment ready for pipeline execution.

CELL 0.5: Bad MOS Filtering (Early PID Culling)

Purpose: Filter out officers with bad MOS codes using `pri_svc_occ_cd` column upfront, before loading snapshot data.

Input: - CSV file: `MOSCULL.csv` (optional, in `./winbucket_link/`) - OR hardcoded list: `['MC', 'DC', 'VC', 'SP', 'AN', 'JA', 'CH', 'MS']`

Algorithm:

1. **Load Bad MOS Codes**:
   - Try to load from CSV file (`MOSCULL.csv`)
   - Fallback to hardcoded list if CSV not found
2. **Create `ref_cull_mos` Table**:
   - SQL table containing bad MOS codes (first 3 characters)
   - Uses PostgreSQL `COPY` command if CSV available (more efficient)
   - Otherwise uses `INSERT` statements
3. **Create `pids_o` Table** (All Officer PIDs):
   - Filters from master table: `rank_grp_pde IN ('OJ','OS')` (commissioned officers)
   - Excludes PIDs starting with 'BAD%' (de-identification errors)
   - Creates distinct list of officer PIDs
4. **Create `ref_cull_pids` Table** (PIDs to Exclude):

- Joins `pids_o` with master table
- Filters where first 3 characters of `pri_svc_occ_cd` match `ref_cull_mos`
- Creates list of PIDs with bad MOS codes

5. **Create `pids_o_culld` Table** (Filtered Officer PIDs):
   - Uses SQL `EXCEPT` operator: `pids_o EXCEPT ref_cull_pids`
   - Final filtered list of officer PIDs (no bad MOS codes)

Output: SQL table `study_talent_net_shared.pids_o_culld`

Key Features: - Early filtering reduces data volume before loading snapshots - Matches logic from SQL scripts (`build_pids_o.sql`, `build_ref_cull_mos.sql`, etc.) - Reports counts at each step (initial PIDs, excluded PIDs, final PIDs) - Handles missing CSV file gracefully with fallback

Note: The 'BAD%' filter excludes PIDs where de-identification failed (PID literally starts with 'BAD').

CELL 1: SQL Table Creation

Purpose: Create SQL tables for efficient data filtering before loading into pandas.

Input: - Master table: `study_talent_net.mv_master_ad_army_qtr_v3a` - Filtered PIDs: `study_talent_net_shared.pids_o_culld` (from CELL 0.5)

Tables Created:

1. `snapshots` **Table**:
   - Distinct snapshot dates from master table
   - Indexed on `snpsht_dt` for performance
2. `snap_window` **Table**:
   - Snapshot dates within specified date window (`window_tup`)
   - Adds fiscal year column (`fy`) for filtering
   - Indexed on `snpsht_dt` and `fy`
3. `b_502_base` **Table** (Base Table):
   - Filters to commissioned officers: `rank_grp_pde IN ('OJ','OS')`
   - Filters to PIDs in `pids_o_culld` table
   - Filters to snapshot dates in `snap_window`
   - Selects columns from `col_list_base`
   - Indexed on `pid_pde` and `snpsht_dt`

Output: SQL table `study_talent_net_shared.b_502_base`

Key Features: - Leverages SQL for fast filtering before pandas - Uses indexed tables for performance - Filters by date window, PID list, and rank group - Selects only needed columns (reduces memory usage)

CELL 2: Load Base DataFrame

Purpose: Load the base DataFrame from the SQL table created in CELL 1.

Input: SQL table `study_talent_net_shared.b_502_base`

Processing: - Loads all rows from SQL table into pandas DataFrame - Reports row count, officer count, and column count

Output: `df_502_00_base.feather`

Key Features: - Efficient loading from SQL using `pd.read_sql()` - Saves intermediate result for troubleshooting - Reports summary statistics

---

CELL 3: Data Quality Checks

Purpose: Identify and exclude officers with data quality issues.

Input: `df_502_00_base`

Checks Performed:

1. **Breaks in Service**:
   - Identifies officers with gaps in snapshot dates
   - Expected: all snapshots between first and last date for each officer
   - Excludes officers with breaks in service
2. **Duplicate Appointment Dates**:
   - Filters to Regular Army only (`compo = 'A'`)
   - Identifies officers with multiple distinct appointment dates
   - Excludes officers with duplicate appointment dates
3. **Duplicate DOR Dates (CPT)**:
   - Filters to CPT rank and Regular Army
   - Identifies officers with multiple distinct CPT DOR dates
   - Excludes officers with duplicate DOR dates
4. **Exclude Problematic Officers**:
   - Combines all problematic PIDs
   - Removes all snapshots for these officers (all ranks)

Output: `df_502_01_clean.feather`

Key Features: - Comprehensive data quality checks - Reports counts for each check (initial, problematic, final) - Excludes entire officers (all their snapshots) if any data quality issue found - Preserves all ranks for remaining officers

---

CELL 4: Calculate Year Groups (yg)

Purpose: Calculate year group (yg) for each officer from their appointment date using fiscal year calculation.

Input: `df_502_01_clean`

Algorithm:

1. **Extract Appointment Dates**:
   - Filters to Regular Army only (`compo = 'A'`)
   - Gets unique appointment dates per officer
2. **Calculate Fiscal Year**:
   - Uses `get_fy()` function to calculate fiscal year from appointment date
   - Fiscal year: October 1 - September 30 (e.g., Oct 1, 2005 = FY 2006)
3. **Handle Multiple Appointment Dates**:
   - Identifies officers with multiple distinct year groups
   - Uses mode (most common) year group for these officers
   - Reports officers with multiple yg values
4. **Map YG to All Snapshots**:
   - Creates yg dictionary (one yg per officer)
   - Maps yg to all snapshots for each officer (all ranks)

Output: `df_502_02_with_yg.feather`

Key Features: - Simplified logic (no complex fallback mechanisms) - Handles multiple appointments by using mode - Maps yg to all snapshots (all ranks) - Reports year group distribution

---

CELL 5: Calculate Date of Rank (dor_cpt and dor_maj)

Purpose: Calculate date of rank (DOR) for CPT and MAJ promotions, mapping to all snapshots.

Input: `df_502_02_with_yg`

Algorithm:

1. **Calculate `dor_cpt`**:
   - Filters to CPT rank ONLY (temporary filter to extract DOR dates)
   - Gets unique CPT DOR dates per officer
   - For officers with multiple DOR dates, uses mode (most common)
   - Maps `dor_cpt` to ALL snapshots (all ranks) - preserves entire career history
2. **Calculate `dor_maj`**:
   - Filters to MAJ rank ONLY (temporary filter to extract DOR dates)
   - Gets unique MAJ DOR dates per officer

- For officers with multiple DOR dates, uses mode (most common)
- Maps `dor_maj` to ALL snapshots (all ranks) - preserves entire career history

3. **Data Quality Check**:
   - Identifies officers where appointment date is after CPT DOR (data quality issue)
   - Excludes these officers (removes ALL their snapshots, all ranks)

Output: `df_502_03_with_dor.feather`

Key Features: - **IMPORTANT**: DOR is calculated from CPT/MAJ snapshots but mapped to ALL snapshots - Preserves entire career history (2LT, 1LT, CPT, MAJ, LTC, etc.) - Allows analysis of career progression (prestige units as LT, job code changes, etc.) - Reports rank distribution to confirm all ranks are present

---

CELL 6: Create Prestige Unit Variables

Purpose: Create time-varying prestige unit variables mapped to all snapshots.

Input: `df_502_03_with_dor`

Algorithm:

1. **Load UIC Hierarchy Data**:
   - Tries to load from standard location: `df_uic_hierarchy.feather`
   - Tries alternative location: `504_build_df_uic_lookup_working.ipynb` output
   - Falls back gracefully if not found
2. **Create Unit Lists**:
   - **Ranger units** (prestige): RGR units from hierarchy
   - **Special Forces units** (prestige): SFG units from hierarchy
   - **SOAR units** (prestige): SOAR units from hierarchy
   - **Combined prestige list**: All prestige units combined
3. **Create `prestige_unit` (Time-Varying Binary)**:
   - 1 if current assignment (`asg_uic_pde`) is in prestige list, 0 otherwise
   - Time-varying: can change at each snapshot
   - Mapped to ALL snapshots (all ranks)
4. **Create `prestige_sum` (Time-Varying Cumulative)**:
   - Cumulative sum of `prestige_unit` by officer
   - Tracks how many quarters/snapshots the officer has been in prestige units
   - Time-varying: increases over time
   - Mapped to ALL snapshots (all ranks)

Output: `df_502_04_with_prestige.feather`

Key Features: - Time-varying variables mapped to all snapshots (all ranks) - Allows analysis of prestige unit service throughout entire career (as LT, CPT, MAJ, etc.) - Reports statistics (officers with prestige service, distribution by rank) - Handles missing UIC

hierarchy data gracefully

CELL 7: Final Output

Purpose: Prepare final dataset and save as `df_502_base.feather` for use in 520 pipeline.

Input: `df_502_04_with_prestige` (or falls back to `df_502_03_with_dor` if prestige not created)

Processing:

1. **Verify Required Columns**:
   - Checks for required columns: `pid_pde`, `snpsht_dt`, `rank_pde`, `yg`, `dor_cpt`, `dor_maj`
   - Checks for optional prestige columns: `prestige_unit`, `prestige_sum`
2. **Final Summary**:
   - Column coverage report
   - Rank distribution (confirms all ranks are preserved)
   - Date range
   - Officer and row counts
3. **Save Final Dataset**:
   - Saves to `df_502_base.feather`
   - This is the output file for use in 520 pipeline

Output: `df_502_base.feather`

Key Features: - Final output ready for Cox regression analysis - **IMPORTANT**: All snapshots are preserved (all ranks) for complete career history - Verifies required columns exist - Reports comprehensive summary statistics

---

Key Features

1. SQL-First Filtering

- Leverages PostgreSQL for efficient early filtering
- Reduces data volume before loading into pandas
- Uses indexed tables for performance
- Early PID culling (CELL 0.5) reduces processing time

2. Progressive Saves

Each stage saves intermediate results using standard naming: - `df_502_00_base.feather` - Base data from SQL - `df_502_01_clean.feather` - After data quality checks - `df_502_02_with_yg.feather` - With year groups - `df_502_03_with_dor.feather` - With dates of rank - `df_502_04_with_prestige.feather` - With prestige variables - `df_502_base.feather` - Final output for 520 pipeline

Benefits: - Can resume from any stage - Can skip early stages if intermediate files exist - Enables troubleshooting at each stage

3. Boolean Cell Flags

Each cell can be enabled/disabled via boolean flags in CELL 0: - Set `CELL0_5 = False` to skip Bad MOS filtering (if table exists) - Set `CELL3 = False` to skip data quality checks (if already done) - Allows selective execution for iterative development

4. Preserve All Snapshots

- Keeps all officer snapshots (all ranks) throughout the pipeline
- No rank-specific filtering (unlike old 501 notebook)
- DOR calculations use CPT/MAJ snapshots to extract dates, but map back to ALL snapshots
- Prestige variables are time-varying and mapped to ALL snapshots
- Enables analysis of career progression (prestige units as LT, job code changes, etc.)

5. Time-Varying Variables

- Prestige variables (`prestige_unit`, `prestige_sum`) are time-varying
- Mapped to all snapshots (all ranks) for complete career history
- Allows analysis of career progression over time

6. Comprehensive Data Quality Checks

- Breaks in service detection
- Duplicate appointment date detection
- Duplicate DOR date detection
- Appointment after DOR detection
- Excludes entire officers (all their snapshots) if any issue found

7. Real-Time Feedback

- Uses `time_start()` and `time_stop()` functions for timing
- Progress reporting for long-running operations
- Nested indentation for sub-operations
- Detailed status messages

Data Flow and Intermediate Files

Input Files

- **Master Data Table:** `study_talent_net.mv_master_ad_army_qtr_v3a` (PostgreSQL)
- **Bad MOS Codes** (optional): `MOSCULL.csv` (in `./winbucket_link/`)
- **UIC Hierarchy** (optional): `df_uic_hierarchy.feather` (for prestige units)

SQL Tables Created

1. `study_talent_net_shared.ref_cull_mos` - Bad MOS codes
2. `study_talent_net_shared.pids_o` - All officer PIDs

3. `study_talent_net_shared.ref_cull_pids` - PIDs with bad MOS
4. `study_talent_net_shared.pids_o_culld` - Filtered officer PIDs
5. `study_talent_net_shared.snapshots` - Distinct snapshot dates
6. `study_talent_net_shared.snap_window` - Snapshot dates in window
7. `study_talent_net_shared.b_502_base` - Base filtered table

Intermediate Files

1. `df_502_00_base.feather` - Base data from SQL
2. `df_502_01_clean.feather` - After data quality checks
3. `df_502_02_with_yg.feather` - With year groups
4. `df_502_03_with_dor.feather` - With dates of rank
5. `df_502_04_with_prestige.feather` - With prestige variables

Output Files

- **Final Output:** `df_502_base.feather` (for use in 520 pipeline)

Configuration and Customization

Date Window

Edit `window_tup` in CELL 0 to change the date range:

```
window_tup = (2000, 2022)  # (start_year, end_year)
```

Column Selection

Edit `col_list_base` in CELL 0 to include/exclude columns:

```
col_list_base = [
    'snpsht_dt', 'pid_pde', 'compo', 'ofcr_apnt_dt',
    'rank_pde', 'ppln_pgrd_eff_dt',
    # ... add or remove columns as needed
]
```

Bad MOS Codes

- Place `MOSCULL.csv` in `./winbucket_link/` directory
- Or modify hardcoded list in CELL 0.5:

```
bad_mos_codes = ['MC', 'DC', 'VC', 'SP', 'AN', 'JA', 'CH', 'MS']
```

Prestige Unit Lists

Prestige units are automatically identified from UIC hierarchy data: - Ranger units (RGR) - Special Forces units (SFG) - SOAR units

To modify prestige unit definitions, edit CELL 6 unit list creation logic.

Dependencies

Required Files

- `functionsG_working.py` - Utility functions (time_start, time_stop, load_feather, store_feather, get_fy, etc.)
- `init_ray_cluster_working.py` - Ray cluster initialization (if using Modin/Ray; `dis_mem_mon` controls memory monitor)

Required Libraries

- `pandas` - Data manipulation
- `numpy` - Numerical operations
- `sqlalchemy` - Database connectivity
- `psycopg2` - PostgreSQL adapter
- `datetime` - Date handling

Optional Files

- `MOSCULL.csv` - Bad MOS codes (in `./winbucket_link/`)
- `df_uic_hierarchy.feather` - UIC hierarchy data (for prestige units)

Usage Workflow

First-Time Run

1. Set all `CELL*` flags to `True` in CELL 0
2. Ensure `MOSCULL.csv` is in `./winbucket_link/` (or use hardcoded list)
3. Ensure UIC hierarchy data is available (optional, for prestige units)
4. Run cells sequentially (CELL 0 → CELL 7)
5. Intermediate files are saved automatically
6. Final output: `df_502_base.feather`

Iterative Development

1. Set flags for cells you want to re-run to `True`
2. Set flags for cells you want to skip to `False`
3. Run notebook - skipped cells will load existing intermediate files
4. Modify configurations (date window, column selection) as needed

Troubleshooting

1. If a cell fails, check the intermediate file from the previous stage
2. Use intermediate save points (e.g., `df_502_02_with_yg.feather`)
3. Enable verbose output (`verbose = True` in CELL 0)
4. Check progress messages and timing information
5. Verify SQL tables exist if skipping CELL 0.5 or CELL 1

Key Differences from 501

1. **Streamlined Structure**: Cleaner, more modular design with better comments
2. **SQL-First Approach**: Leverages PostgreSQL for efficient early filtering
3. **Preserves All Snapshots**: No rank-specific filtering (501 filtered to CPT only)
4. **Progressive Saves**: Standard naming pattern for intermediate files
5. **Better Documentation**: Comprehensive comments and logical flow
6. **Early PID Culling**: Bad MOS filtering happens upfront (CELL 0.5)
7. **Time-Varying Prestige**: Prestige variables mapped to all snapshots
8. **Simplified Logic**: Removed over-complicated fallback mechanisms
9. **Data Quality Focus**: Comprehensive checks with clear reporting
10. **Integration Ready**: Output designed for use in 520 pipeline

---

Integration with 520 Pipeline

The `502_working.ipynb` notebook is designed to be a preprocessor for `520_pipeline_cox_working.ipynb`:

What 502 Provides

- **Filtered Data**: Commissioned officers only, bad MOS filtered out
- **Data Quality**: Officers with data quality issues excluded
- **Year Groups**: `yg` column calculated from appointment date
- **Dates of Rank**: `dor_cpt` and `dor_maj` columns calculated
- **Prestige Variables**: `prestige_unit` and `prestige_sum` (time-varying)
- **All Snapshots**: Complete career history (all ranks) preserved

How 520 Uses 502 Output

In `520_pipeline_cox_working.ipynb` CELL 1:

```
load_from_502 = True   # Set to True to load from 502 output (recommended)

if load_from_502:
    df_raw = load_feather('df_502_base')
    # 502 output already includes:
    # • Commissioned officers filtered
    # • Data quality checks completed
    # • Year group (yg) calculated
    # • Date of rank CPT (dor_cpt) calculated
    # • Date of rank MAJ (dor_maj) calculated
```

Benefits: - 520 can skip redundant filtering (CELL 2) - 520 can skip year group calculation (CELL 3) - Faster pipeline execution - Consistent data preparation

---

Summary

The `502_working.ipynb` notebook provides an efficient, SQL-based data preparation pipeline that:

- **Leverages SQL** for fast early filtering and PID culling
- **Preserves all snapshots** (all ranks) for complete career history
- **Performs comprehensive data quality checks** to ensure clean data
- **Calculates key demographic variables** (year groups, dates of rank, prestige units)
- **Uses progressive saves** for selective execution and troubleshooting
- **Integrates seamlessly** with the 520 Cox regression pipeline

The pipeline is designed to be modular, efficient, and user-friendly, providing a solid foundation for Cox regression analysis of military officer promotion timing.